

LangGraph Session Notes

Encoding → Agents → LangGraph → Memory · Yash Jaiswar

1 · Encoding & Embeddings

1.1 Core Distinction

Encoding = converting text to a **fixed numerical representation** (one-hot, BOW, TF-IDF). No semantic meaning across words.

Embedding = a **dense learned vector** capturing semantic relationships. Similar words sit close in vector space.

1.2 Encoding Methods

Method	What it produces	Key weakness
One-Hot	Binary vector, 1 at word index	No semantics; huge sparse vector
Bag of Words (BOW)	Word frequency counts	Loses word order
TF-IDF	Weighted importance score	Still sparse; no context
Word2Vec	Dense semantic vector (static)	Same vector for all contexts

1.3 TF-IDF Formula

$TF(t,d) = \text{count of } t \text{ in doc } d / \text{total terms in } d$

$IDF(t) = \log(N / df(t))$ — N = total docs, $df(t)$ = docs containing t

$TF-IDF(t,d) = TF \times IDF$ — high score = frequent here, rare overall.

1.4 Static vs Contextual Embeddings

- **Word2Vec (static):** 'bank' always has the same vector regardless of context.
- **Transformers (contextual):** 'bank' near 'river' $\neq$ 'bank' near 'loan' — vector shifts with context.
- This is the core problem Word2Vec cannot solve and why Transformers replaced it.

1.5 CountVectorizer — fit_transform

`fit_transform(corpus)` does two things: learns vocabulary (fit) then vectorises (transform).

On new data: use **transform only** — never re-fit on test data.

2 · Transformer & Agent Intuition

2.1 Why Transformers Over Word2Vec

- Word2Vec: one static vector per word. Cannot handle polysemy (multiple meanings).
- Transformer self-attention produces a different vector for the same word depending on surrounding tokens.

2.2 Agentic AI — The Agent Loop

Step	What happens
------	--------------

1. Perceive	Agent receives goal/input
2. Think	LLM reasons about what action to take
3. Act	Calls a tool or produces output
4. Observe	Tool result fed back into context
5. Repeat	Loop until goal met or LLM calls END

2.3 ReAct Pattern

ReAct = Reasoning + Acting. The LLM interleaves thought steps with tool calls:

- Thought → Action → Observation → Thought → ... → Final answer

3 · LangGraph Fundamentals

3.1 Core Primitives

Node — a Python function. Receives state, does one unit of work, returns updated state.

Edge — defines control flow. Two types:

- **Normal edge:** always goes to the same next node (compile-time)
- **Conditional edge:** chooses next node based on state (e.g. tools_condition)

State — TypedDict or Pydantic model flowing through the graph.

3.2 Three Ways to Create Tools

Method	When to use
@tool decorator	Simplest — wrap any Python function. Docstring = tool description.
StructuredTool.from_function()	When you need explicit Pydantic input schema.
BaseTool subclass	Full control — custom run(), arun(), error handling.

3.3 tools_condition

Conditional edge used after the LLM node. Inspects the **last message in state**:

- Has **tool_calls** field → route to ToolNode
- No tool_calls → route to END

This creates the ReAct loop inside a LangGraph graph.

4 · Memory & Checkpointing

4.1 Memory Types

Type	Storage	Purpose
Short-term	Redis (rpush/ltrim/lrange)	Sliding window of recent messages
Long-term	SQLite	Persistent facts across sessions
Semantic	Qdrant (vector DB)	Similarity-based retrieval of past context

RAG knowledge	Qdrant (384-dim, cosine)	Domain knowledge for grounded answers
---------------	--------------------------	---------------------------------------

4.2 MemorySaver + thread_id

MemorySaver saves the full graph state after every node execution.

thread_id = the key for a specific conversation's checkpoint. Each unique thread_id = isolated memory slot. LangGraph uses it to restore the entire graph state — not just chat history.

4.3 Common Mistakes

■■ MISTAKE 1: Using the string 'add_messages' instead of the function

WRONG: messages: Annotated[list, 'add_messages']

RIGHT: messages: Annotated[list, add_messages]

The reducer must be the imported function — string silently fails.

■■ MISTAKE 2: Generating thread_id inside the loop

WRONG: config = {'configurable': {'thread_id': str(uuid4())}} # inside loop

RIGHT: thread_id = str(uuid4()) # once, BEFORE the loop

New UUID each call = new thread each call = no memory continuity.

5 · Human-in-the-Loop (HITL)

5.1 How interrupt() Works

interrupt() pauses the graph mid-execution and waits for human input.

- invoke() #1 → graph runs until interrupt() → pauses → returns to caller
- Human provides approval/input
- invoke() #2 with Command(resume=human_input) → resumes from checkpoint

■ Why MemorySaver is REQUIRED for HITL:

interrupt() saves graph state as a checkpoint. The second invoke() with Command(resume=...) loads that checkpoint to know exactly where to resume.

Without MemorySaver → no checkpoint → second invoke() raises an ERROR.

The resume mechanism breaks entirely. Not just missing history — the graph crashes.

6 · Send API & Map-Reduce

6.1 Send vs Regular Edges

	Regular Edge	Send API
Defined at	Compile time (graph.compile())	Runtime (inside a node)
Branches	Fixed — same every run	Dynamic — decided during execution

Parallelism	No (sequential)	Yes — fan-out to N workers simultaneously
Use case	Known linear flow	Map-reduce, orchestrator-worker

6.2 Annotated[list, operator.add] — Fan-in Reducer

When Send dispatches N workers in parallel, each writes to the same state field.

Without reducer: parallel writes **overwrite each other** — only last result survives.

With **Annotated[list, operator.add]**: each worker's output is **appended**. All N results collected.

6.3 Challenge 5 — Orchestrator-Worker Blueprint

Component	Role	Key detail
Orchestrator node	Generate 3 subtopics from user subject	Returns list of subtopics
Send dispatch	Fan-out — one Send per subtopic	from langgraph.types import Send
Worker node	Research one subtopic, write paragraph	Different state schema
Combiner node	Merge all 3 paragraphs into final report	Receives Annotated list
State reducer	Accumulate worker outputs	Annotated[list, operator.add]

7 · 6 LangGraph Workflow Patterns

Pattern	Description	Key mechanism
Sequential	A → B → C, linear pipeline	Normal edges
Conditional	Branch based on state	Conditional edge + router function
Fan-out / Fan-in	One node → N parallel workers → collect	Send + Annotated[list, operator.add]
Orchestrator-Worker	Orchestrator assigns tasks at runtime	Send from orchestrator node
Evaluator-Optimizer	Generate → score → loop if poor	Conditional edge back to generator
ReAct Loop	Reason → tool call → observe → repeat	tools_condition edge

8 · Subgraphs

Subgraphs allow you to embed one compiled graph as a node inside another.

Scenario	How to integrate
Shared state schema with parent	Add subgraph directly: graph.add_node('sub', subgraph)
Different state schema	Wrap in a function that transforms state in/out; add the function as a node

9 · Practicals Summary

#	What was built	Key concepts
1	Agent loop — goal → LLM plans 3 steps → executes each	Groq API, loop, context passing
2	Memory chatbot with Redis persistence	MessagesState, MemorySaver, thread_id, add_messages

3	Tool-calling agent (Tavily + custom tool)	@tool, bind_tools, ToolNode, tools_condition
4	HITL approval agent	interrupt(), Command(resume=...), MemorySaver, same thread_id
5	Orchestrator-Worker with Send API	NOT YET STARTED — next challenge

10 · Corrections from This Session

■ HITL + MemorySaver — 'it just won't store anything'

CORRECTION: Without MemorySaver the graph CRASHES on the second invoke(). interrupt() requires a checkpoint to resume from. Command(resume=...) raises an error if no checkpoint exists. Not a silent miss — a hard failure.

■ thread_id — 'stores the chat session'

CORRECTION: thread_id is the key for the full graph state checkpoint — not just chat history. LangGraph restores the entire graph state (nodes, messages, all fields) from the exact moment it paused. Missing this distinction means missing how HITL works.

■ Annotated[list, operator.add] — 'accumulator for final answer'

Correct direction. Precise: prevents parallel worker writes from overwriting each other. Each worker's output is appended, not replaced.

■ Send vs regular edges — 'predefined vs dynamic'

Correct. Precise: regular edges = compile-time (fixed at graph.compile()). Send = runtime — orchestrator decides during execution how many workers and what each gets.

11 · Next Steps

Challenge 5 — Orchestrator-Worker with Send API

- Ref: <https://langchain-ai.github.io/langgraph/how-tos/map-reduce/>
- Orchestrator generates 3 subtopics → Send dispatches each to a worker in parallel
- Each worker writes a research paragraph
- Combiner merges all 3 → final report
- Use Annotated[list, operator.add] as state reducer
- Orchestrator and worker MUST have different state schemas

Assignment 06 — Memory-Augmented RAG → COMPLETED ■

Save to: ai-engineering-practice/notes/LangGraph_Session_Notes.pdf